LAB 4

MỤC TIÊU

- ✓ Thực hiện xác thực và phân quyền
- ✓ Xây dựng tiện ích FileUtil
- ✓ Xây dựng chức năng đăng nhập
- ✓ Xây dựng chức năng quản lý danh mục đồ uống

NỘI DUNG

PHẦN I: PHÂN QUYỀN VÀ ĐĂNG NHẬP

BÀI 1 (4 ĐIỂM): **Thực hiện xây dựng Phân quyền và Xác thực tại Program.cs**

```
using Microsoft.AspNetCore.Authentication.Cookies;
using QuanLyQuanCaPhe.Models;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddTransient<UserDAL>();
// Xác thực: dựa trên cookie
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationSchem
e)
  .AddCookie(options =>
  {
    options.LoginPath = "/Login/Login";
    options.AccessDeniedPath = "/Home/AccessDenied";
    //Xác định thời gian tồn tại của cookie là 24 giờ
    options.ExpireTimeSpan = TimeSpan.FromHours(24);
```

```
    });
// Phân quyền: các policy cho vai trò
builder.Services.AddAuthorization(options =>
{
    // policy cho phép cả Employee và Manager (dùng khi chấp nhận một trong hai)
    options.AddPolicy("EmployeeOrManager", policy => policy.RequireRole("Employee",
"Manager"));
    // policy chỉ cho phép Manager
    options.AddPolicy("ManagerOnly", policy => policy.RequireRole("Manager"));
});
var app = builder.Build();
// Cấu hình pipeline xử lý HTTP.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // Giá trị HSTS mặc định là 30 ngày.
    //app.UseHsts();
}
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();

// QUAN TRỌNG: Xác thực phải được gọi trước Phân quyền
app.UseAuthentication();
app.UseAuthorization();
app.MapControllerRoute(
    name: "default",
```

```
  pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```

BÀI 2 (3 ĐIỂM): Xây dựng chức năng đăng nhập

```
public class LoginController : Controller
{
    private readonly UserDAL _userDal;

    public LoginController(UserDAL userDal)
    {
        _userDal = userDal;
    }

    [HttpGet]
    [AllowAnonymous]
    public IActionResult Login(string returnUrl = null)
    {
        ViewData["Title"] = "Login";
        var model = new LoginViewModel
        {
            ReturnUrl = returnUrl ?? Url.Content("~/")
        };
        return View("Views/Account/Login.cshtml", model);
    }

    [HttpPost]
    [AllowAnonymous]
```

```
  public async Task<IActionResult> Login(LoginViewModel model, string returnUrl =
null)
  {
    model.ReturnUrl = returnUrl ?? model.ReturnUrl ?? Url.Content("~/");

    if (!ModelState.IsValid)
    {
      return View("Views/Account/Login.cshtml", model);
    }

    var user = UserDAL.Authenticate(model.Email ?? string.Empty, model.Password ??
string.Empty);
    if (user != null)
    {
      var roleName = user.Role ? "Manager" : "Employee";

      var claims = new[]
      {
        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new Claim(ClaimTypes.Name, user.FullName ?? user.Email ?? string.Empty),
        new Claim(ClaimTypes.Email, user.Email ?? string.Empty),
        new Claim(ClaimTypes.Role, roleName)
      };

      var identity = new ClaimsIdentity(claims,
CookieAuthenticationDefaults.AuthenticationScheme);
      var principal = new ClaimsPrincipal(identity);
```

```
        var authProperties = new AuthenticationProperties
        {
            IsPersistent = model.RememberMe
        };

        await
HttpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme,
principal, authProperties);

        return LocalRedirect(model.ReturnUrl);
    }

    ModelState.AddModelError(string.Empty, "Invalid email or password.");
    return View("Views/Account/Login.cshtml", model);
  }

  [HttpPost]
  [ValidateAntiForgeryToken]
  [Authorize]
  public async Task<IActionResult> Logout()
  {
    await
HttpContext.SignOutAsync(CookieAuthenticationDefaults.AuthenticationScheme);
    return RedirectToAction("Index", "Home");
  }
}
```

PHẦN II: DANH MỤC ĐỒ UỐNG CRUD CƠ BẢN

BÀI 2 (3 ĐIỂM): Xây dựng các chức năng xem, thêm, sửa, xóa (CRUD) cho Danh mục đồ uống

```
public class CategoriesController : Controller
{
    [Authorize(Policy = "EmployeeOrManager")]
    public IactionResult Index()
    {
        …
    }

    [Authorize(Policy = "EmployeeOrManager")]
    public IactionResult Details(int id)
    {
        …
    }

    [Authorize(Policy = "ManagerOnly")]
    public IactionResult Create()
    {
        …
    }

    [HttpPost]
    [Authorize(Policy = "ManagerOnly")]
    public IactionResult Create(CategoryDAL category)
    {
```

```
    ...
}

[Authorize(Policy = "ManagerOnly")]
public IactionResult Edit(int id)
{
    ...
}

[HttpPost]
[Authorize(Policy = "ManagerOnly")]
public IactionResult Edit(int id, CategoryDAL category)
{
    ...
}

[Authorize(Policy = "ManagerOnly")]
public IactionResult Delete(int id)
{
        ...
}

[HttpPost]
[Authorize(Policy = "ManagerOnly")]
public IactionResult DeleteConfirmed(int id)
{
    ...
```

```
  }
}
```

*** YÊU CẦU NỘP BÀI:

Sinh viên nén file bao gồm các yêu cầu đã thực hiện trên, nộp LMS đúng thời gian quy định của giảng viên. Không nộp bài xem như không có điểm.

--Hết--